RECORD

Course:	Full Stack Web Development	**Course Code:**	23CM4121
Experiment No.:	5	**Date:**	30-08-2026
Student Name:	R Varshini	**Roll No.:**	A24126552115

1. TITLE

Implement Modules in Node.js

2. AIM

To understand and implement different types of modules in Node.js — custom (user-defined) modules, built-in (core) modules, and ES modules — and to demonstrate how each type is created, exported, and imported.

3. OBJECTIVE

- To understand the concept of modules in Node.js.
- To create and export a custom module using module.exports.
- To import and use a custom module using require().
- To work with built-in Node.js modules such as fs and os.
- To read from and write to files using the File System (fs) module.
- To retrieve system information using the OS module.
- To understand and implement ES Modules using import and export.

4. THEORY

A module in Node.js is a reusable block of code whose functionality can be exported and used in other files. Node.js follows a modular architecture, and modules are broadly classified into three types:

- Custom Modules: User-defined modules created by the developer, exported using module.exports and imported using require().
- Built-in (Core) Modules: Modules that come pre-packaged with Node.js, such as fs (File System) and os (Operating System), and do not require separate installation.
- ES Modules: Modules based on the ECMAScript standard that use import and export statements instead of require() and module.exports.

The fs module allows reading from and writing to files on the system, while the os module provides information about the underlying operating system such as platform, architecture, CPU details, and memory usage.

The basic flow of a modular Node.js application is:

Module Definition → module.exports / export → require() / import → Usage in Main File

5. ALGORITHM / PROCEDURE

1. Create a custom module file (node.js) with functions to add and subtract two numbers.
2. Export the functions from the custom module using module.exports.
3. Create a main file (node1.js) and import the custom module using require().
4. Call the imported functions and display the results using console.log().
5. Create another file (builtin.js) to explore built-in modules.
6. Import the fs module and use writeFileSync() and readFileSync() to write and read a text file.
7. Import the os module and use its methods to display platform, architecture, CPU, and memory details.
8. Create an ES module file (node2.js) that exports a greet function using export default.
9. Create a main file (node3.js) that imports the greet function using ES Module import syntax.
10. Run each file using Node.js and verify the output in the terminal/debug console.
11. Record and compare the actual output with the expected output.

6. PROGRAM

node.js (Custom Module)

```
function add(a, b) {
    return a + b;
}

function subtract(a, b) {
    return a - b;
}

module.exports = {
    add,
    subtract
};
```

node1.js (Importing Custom Module)

```
const math = require('./node');

console.log(math.add(10, 5));
console.log(math.subtract(10, 5));
```

builtin.js (Built-in Modules — fs and os)

```
const fs = require('fs');

fs.writeFileSync('hello.txt', 'Hello Node');

const data = fs.readFileSync('hello.txt','utf8');

console.log(data);
//builtin
const os = require('os');
```

```
console.log(os.platform());
console.log(os.arch());
console.log(os.cpus());
console.log(os.totalmem());
console.log(os.freemem());
```

node2.js (ES Module — Export)

```
export default function greet(name) {
    return `Hello, ${name}!`;
}
```

node3.js (ES Module — Import)

```
import greet from './node2.js';

console.log(greet("Alice"));
```

7. CODE EXPLANATION

Code	Explanation
`require('./node')`	Imports a custom local module (node.js) using its relative path.
`module.exports`	Exposes selected functions/values from a module so other files can use them.
`fs.writeFileSync()`	Writes data to a file synchronously using the built-in File System module.
`fs.readFileSync()`	Reads data from a file synchronously and returns its contents.
`require('os')`	Imports the built-in OS module to access system information.
`os.platform()` / `os.arch()`	Returns the operating system platform and CPU architecture.
`os.cpus()` / `os.totalmem()` / `os.freemem()`	Returns CPU core details, total memory, and free memory of the system.
`export default`	Marks a function as the default export of an ES module.
`import greet from './node2.js'`	Imports the default export from an ES module using ES Module syntax.

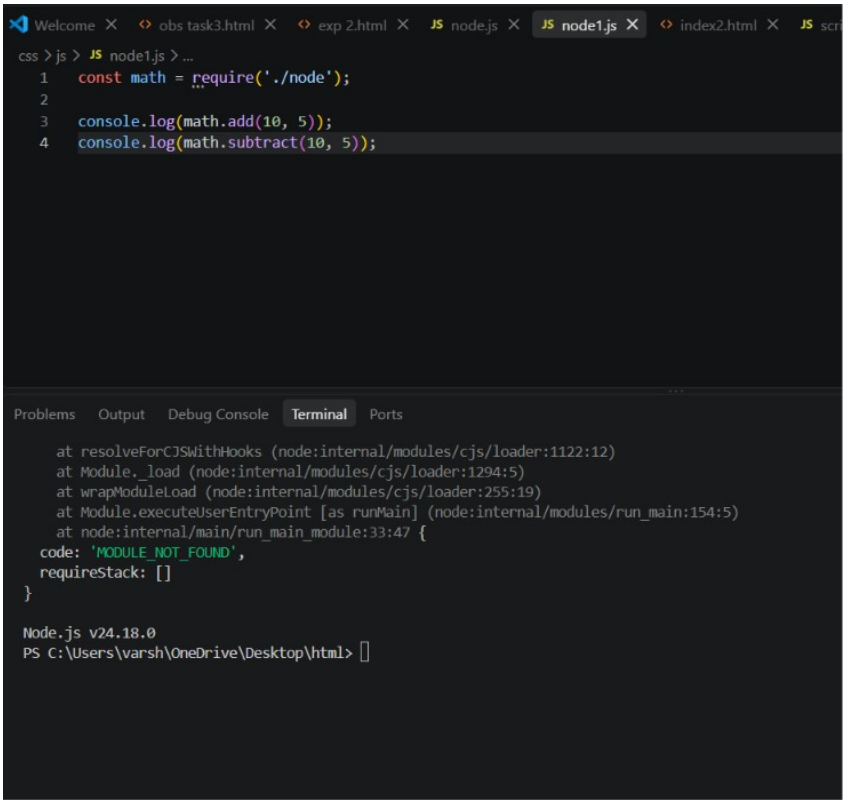
8. TEST CASES AND EXECUTION DATA

TC-01	node1.js	add(10,5), subtract(10,5)	Displays 15 and 5	Pass
TC-02	builtin.js	Write & read hello.txt	Displays 'Hello Node' and system info	Pass

TC-03	builtin.js	os.platform(), os.arch()	Displays win32 and x64	Pass
TC-04	node3.js	greet("Alice")	Displays 'Hello, Alice!'	Pass

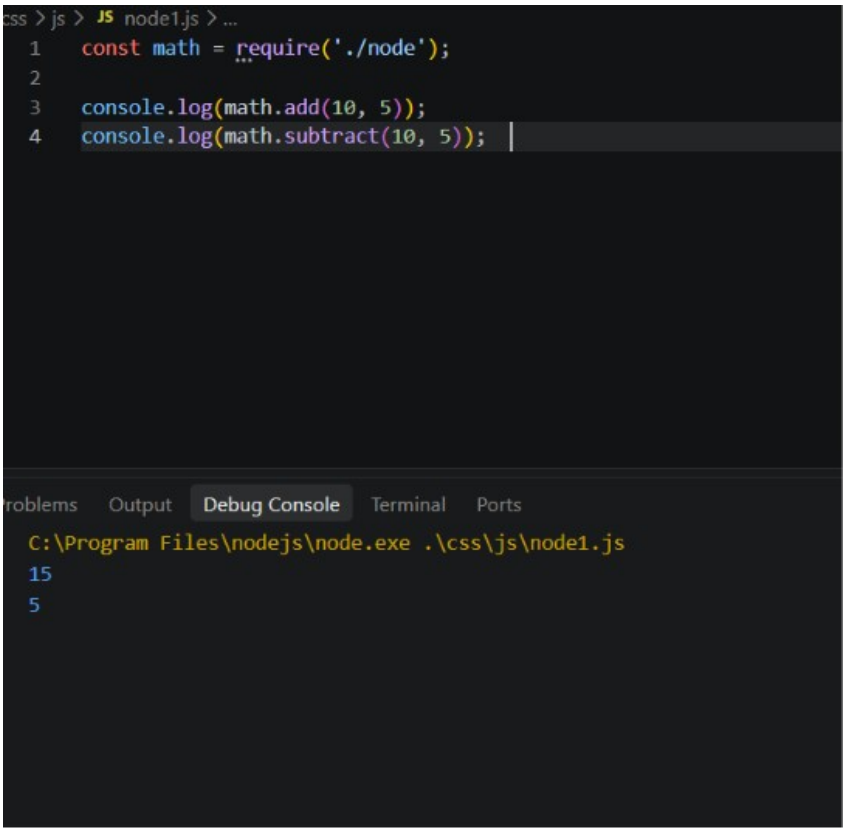
9. OUTPUT

Output 1: Custom Module — Import Error Before Fix



Initial run using an incorrect entry point resulted in a MODULE_NOT_FOUND error.

Output 2: Custom Module — add() and subtract()

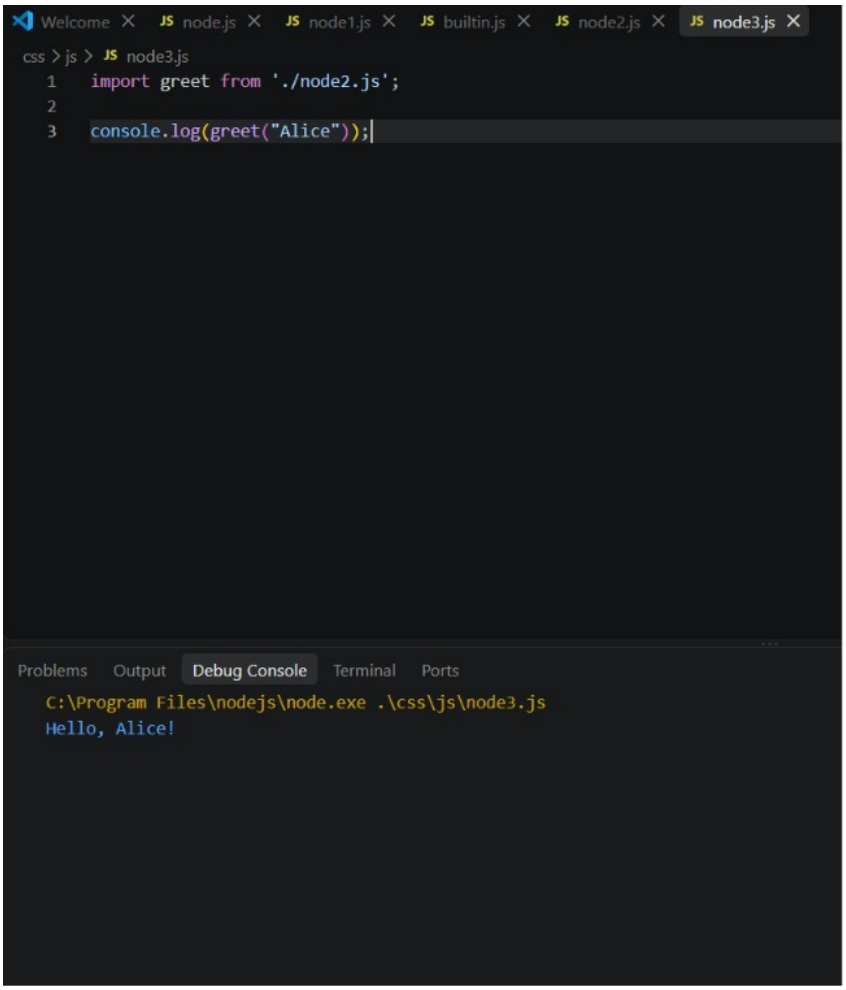


node1.js correctly imports node.js and prints 15 and 5.

Output 3: Built-in Modules — fs and os

builtin.js writes and reads hello.txt, then prints platform, architecture, CPU list, total memory, and free memory using the os module.

Output 4: ES Modules — import / export



node3.js imports greet from node2.js using ES Module syntax and prints 'Hello, Alice!'.

10. RESULT

Thus, different types of modules in Node.js — custom modules, built-in modules (fs and os), and ES modules — were successfully implemented and executed. The functions were imported and used correctly, and the outputs were verified against the expected results.